

Regular expressions are used for pattern matching

to design regular expression we use following symbols

.	it matches with 1 character
[a-zA-Z]	it matches with any alphabet
[0-9] or \d	It matches with single digit
\w	it matches with any one word character [a-zA-Z0-9_]
\W	it matches with any one word character other than alphabet, digits and underscore [^a-zA-Z0-9_]
\D	any non digit character
\b	any one boundary character
\B	any one non boundary character
\s	it matches with space
\S	it matches with any non-space character
*	0 or more occurrences
+	1 or more occurrences
?	0 or 1 occurrence
{n}	exactly n occurrences
{m,n}	minimum m occurrences, maximum n occurrences
{m,}	minimum m occurrences
^	it matches the pattern at the beginning of the string
\$	it matches the pattern at the end of the string

s	[Oo]r	^[Oo]r	[Oo]r\$	\b[Oo]r\b
This is origami	y	N	N	N
This is normal	y	N	N	N
There is a tailor	y	N	Y	N
this or that	y	N	n	Y
Or	y	y	Y	Y
There is a cat	N	N	N	N
Origami is good	y	Y	N	N

S="Something is there somewhere"

[Ss].*e	Something is there somewhere		
[sS].*?e	Some		
\w+\s\w+\s\w+	Something is there somewhere		
^\w+\s\w+\s\w+\$	This is line		

functions in re module

<code>re.search(pattern,string,flags)</code>	it will find the first occurrence of the given pattern, anywhere in the string, and returns a match object
<code>re.match(pattern,string,flags)</code>	it will find the first occurrence of the given pattern, at the beginning in the string, and returns a match object
<code>re.findall(pattern,string,flags)</code>	It will find all occurrences of the pattern in the string and returns a list of strings
<code>re.finditer(pattern,string,flags)</code>	It will find all occurrences of the pattern in the string and returns a list of match objects
<code>re.compile(pattern,flags)</code>	It returns a regular expression object, and stores pattern and flags in it
<code>re.sub(pattern,newstr,s,count=0,flags=re.I re.M)</code>	It will substitute all occurrences of the given pattern, by newstr but if count is given , then it will replacecount number of occurrences with the given string

Object oriented programming

To keep data and functions/methods as a single unit, we use Classes

1. Within class members can be instance variables or static variables
 - a. Instance variable--→ member variables for which you need separate memory for every object , then it is called as instance variables
 - b. Staic variable--→ member variable whose memory is shared by all objects of that class is called as static variables
2. Within a class we can write functions
 - a. Constructor --→ it initializes the object(allocate memory and initialize values)
 - b. Accessor and mutators--→ all setter methods allows to modify values of a member variable, after creation of object, these are called as mutator methods. Getter methods allows to access value of one of the member, is called as accessor methods
If you want to keep any property/member variable read only, then do not add setter method for that member variable/property
 - c. Member methods--→ any other function written inside class, other than constructor/ setter / getter methods, then it is called as member method/function

Within a class all variables / functions whose name starts with `__` are private members of the class, and will not be accessible outside the class

If the name starts with `_` then, it is protected members of the class

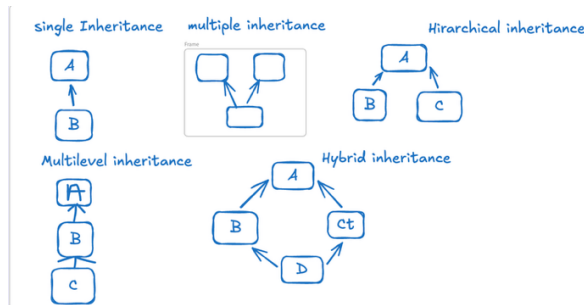
Usually protected members are accessible only within a class and within a child class, but in python protected members are accessible outside the class also

Static members and functions

1. static method can access only static members, it cannot access instance variables
2. static variables are declared inside class without using self.
3. These variables can be accessed by using `classname.variablename`

4. Static methods need to be decorated with @staticmethod
5. If a method inside class has no parameters, then by default the method is static method

Inheritance



file handling

<code>fh=open("mytetx.txt")</code>	open the file in read mode and returns the handle
<code>fh1=open("mytetx.txt","w")</code>	open the file in write mode and returns the handle, if file exists it will overwrite the file, otherwise it creates the files
<code>fh.close()</code>	I closes the file
<code>fh.readlines()</code>	will read all lines from the file and store it in a list
<code>fh.read()</code>	will read all data from the file and store it in a string
<code>fh.read(n)</code>	it reads n characters from current position
<code>fh.seek(n)</code>	moves the poiter at n th position
<code>fh.tell()</code>	gives the current pointer position